

Prompt Engineering, yapay zeka modelleriyle etkileşimde sadece bir komut dizisi oluşturmanın ötesinde, belirli değişkenlere göre optimize edilmesi gereken dinamik bir süreçtir. Bu süreç; kullanılan modelin varsayılan davranışına, zamana bağlı güncellemelere (RLHF süreçleri), etkileşim kurulan dile ve teknik terminolojiye göre farklılık gösterir.

1. Bir Zanaat Olarak Prompt Engineering

Prompt Engineering, katı matematiksel modellere veya değişmez bilimsel formüllere dayanmaz; bunun yerine **sistematik bir deneme-yanılma süreci** üzerine kuruludur. Bu yönüyle bir "mühendislik"ten ziyade, deneyimle gelişen bir **zanaat** (craft) olarak kabul edilir. Geliştiricilerin bu alanda yetkinleşmesi, model davranışlarını sürekli test etmelerine ve elde edilen sonuçlara göre girdileri rafine etmelerine bağlıdır.

2. OpenAI Tarafından Önerilen 6 Temel Strateji

Sektör standartlarını belirleyen OpenAI ekibi, GPT-3.5 ve GPT-4 serisi modellerden maksimum verim alabilmek için süreci 6 ana strateji altında toplamıştır. Bu stratejiler, hem modelin performansını artırmayı hem de çıktı güvenilirliğini sağlamayı hedefler.

01

Prompt Engineering
Girdi Mühendisliği

OPENAI KILAVUZU



OPENAI'S 6 STRATEGIES for
PROMPT ENGINEERING

OPENAI'S 6 STRATEGIES for PROMPT ENGINEERING



- 1 Açık ve anlaşılır talimatlar verin
- 2 Referans metinler verin
- 3 Karmaşık görevleri alt görevlere bölün
- 4 Modele "düşünmesi" için zaman tanıyın
- 5 Modele harici araçlar kullanırın
- 6 Değişiklikleri sistematik biçimde test edin

<https://platform.openai.com/docs/guides/prompt-engineering>

2.1. Açık ve Anlaşılır Talimatlar Vermek

Modelden istenen çıktı ne kadar detaylı ve net tanımlanırsa, hata payı o kadar azalır. Bu strateji kapsamında; modele belirli bir kişilik (persona) kazandırmak, detaylı kısıtlar koymak ve istenen formatı (örneğin JSON veya Markdown) açıkça belirtmek kritik taktikler arasındadır.

2.2. Referans Metinler Kullanmak

Modeller, özellikle GPT serisi, kendilerine sunulan bağlam (context) içerisinden bilgi süzme konusunda oldukça başarılıdır. "Halüsinasyon" riskini azaltmak için modele tarif

yapmak yerine, doğrudan alıntı yapılabilir veya örnek alınabilecek **referans metinler** sunulmalıdır.

2.3. Karmaşık Görevleri Alt Görevlere Bölmek

Muhakeme gerektiren veya çok adımlı işlemlerde, görevi tek bir prompt ile çözmek yerine daha küçük ve yönetilebilir parçalara ayırmak gerekir. Bu yöntem, her bir adımın model tarafından daha doğru işlenmesini sağlar.

2.4. Modele "Düşünmesi" İçin Zaman Tanımak

Modelin doğrudan bir yanıt üretmeye odaklanması yerine, yanıtı formüle etmeden önce bir iç diyalog (**inner monologue**) veya adım adım çözüm süreci işletmesi sağlanmalıdır. Prompt içerisine eklenen "adım adım düşün" veya "acele etme" gibi ifadeler, modelin ara tokenlar üreterek daha nitelikli bir planlama yapmasına yardımcı olur.

2.5. Harici Araçlardan Faydalanmak

Modelin kendi eğitim veri setindeki kısıtları aşmak için dış dünyayla bağlantı kurması sağlanmalıdır. Bu araçlar şunları içerebilir:

- **Web Arama:** Güncel bilgi erişimi için.
- **Bellek Genişletme (RAG):** Spesifik doküman setlerinden bilgi çekmek için.
- **Özel Fonksiyonlar:** Ses sentezleme veya görüntü oluşturma gibi modelin doğal yeteneği olmayan görevlerin harici fonksiyonlara atanması.

2.6. Değişiklikleri Sistemik Biçimde Test Etmek

Prompt optimizasyonu sırasında yapılan her değişikliğin bir önceki versiyondan daha iyi sonuç verip vermediği nesnel kriterlerle ölçülmelidir. Gelişi güzel güncellemeler yerine, sonuçların maliyet, hız ve doğruluk açısından takibi yapılmalıdır.

3. Uygulama ve Test Ortamı: OpenAI Playground

Prompt stratejilerini somutlaştırmak ve sistematik testleri gerçekleştirmek için en verimli ortamlardan biri **OpenAI Playground**'dur. Bu platform üzerinden farklı modeller seçilebilir ve "System Message" katmanında modele karakter analizi veya üslup tanımlamaları yapılabilir.

Teknik İpucu: Token yönetimi maliyet ve hız üzerinde belirleyicidir. Yapılan optimizasyonlarda $\$1 \text{ \textit{Token}} \approx 0.75 \text{ \textit{kelime}}$ formülü göz önünde bulundurularak, çıktı kalitesinden ödün vermeden en az token harcayan prompt yapısı hedeflenmelidir.